Project Title: Simple RAG System with XLS Data and Web Upload

Objective: Develop a basic Retrieval-Augmented Generation (RAG) system in Python that utilizes data from an uploaded Excel (XLS/XLSX) file for retrieval and answers user queries. The system should include a simple web interface for uploading the Excel file.

Deliverables:

- **Python RAG Core:**
    - **Data Ingestion:** A Python script to read data from an uploaded Excel file (XLS/XLSX). Pandas is recommended for this.
    - **Text Chunking:** Implement a method to split the Excel data (e.g., specific columns or rows converted to text) into manageable chunks suitable for embedding.
    - **Embeddings:** Generate vector embeddings for the text chunks using a pre-trained embedding model (e.g., from Hugging Face or OpenAI).
    - **Vector Store:** Store the generated embeddings in a vector database (e.g., FAISS, or a simple in-memory vector store).
    - **Retrieval:** Implement a retrieval mechanism to find relevant text chunks based on a user query.
    - **Generation:** Integrate a Language Model (LLM) to synthesize an answer based on the retrieved chunks and the user's query.
- **Web Interface:**
    - **File Upload:** Create a simple web screen (using Flask, Streamlit, or similar) that allows users to upload an Excel file.
    - **Query Input:** Provide an input field for users to type their questions.
    - **Response Display:** Display the RAG system's generated answer to the user query.

Technical Requirements:

- **Python Version:** Python 3.10+
- **Libraries (Recommended):**
    - pandas for Excel data handling.
    - langchain or llama_index for RAG framework components (document loading, chunking, embeddings, retrieval, LLM integration).

    - sentence-transformers or openai (or other LLM provider's library) for embeddings.
    - faiss-cpu or in-memory for vector storage.
    - flask or streamlit for the web interface.
- **API Keys:** If using external LLMs or embedding models (e.g., OpenAI), ensure proper handling of API keys (e.g., using environment variables).

Instructions:

- **Environment Setup:**

Create a virtual environment and install all necessary Python libraries.

- **Web Application Development:**
    - Design the web interface with an upload button for XLS/XLSX files.
    - Implement the backend logic to handle file uploads, save the file temporarily, and trigger the RAG process.
- **RAG System Implementation:**
    - **Load Data:** Upon file upload, use Pandas to read the Excel data into a DataFrame.
    - ***Prepare Data for RAG:*** *Convert relevant DataFrame content into a format suitable for text chunking (e.g., concatenating columns, extracting specific cells).*
    - **Chunking and Embedding:** Chunk the prepared text and generate embeddings, storing them in your chosen vector store.
    - **Query Handling:** When a user submits a query via the web interface:
        - Generate an embedding for the query.
        - Retrieve the most relevant chunks from the vector store.
        - Pass the retrieved chunks and the query to an LLM to generate a comprehensive answer.
    - **Display Results:** Present the LLM's answer on the web interface.
- **Error Handling:**

Implement basic error handling for file uploads, invalid file formats, and potential issues during the RAG process.

- **Documentation:**

Provide clear comments in your code and a brief README.md explaining how to set up and run the project.

Evaluation Criteria:

- Functionality of the RAG system (accuracy of retrieval and generation).
- User-friendliness and functionality of the web upload interface.
- Code quality, readability, and adherence to best practices.
- Proper handling of Excel data.
- Clear documentation.